AI-Mediated Personal Infrastructure

From AI-Built Apps to AI-Coordinated Personal Systems

Omica Chow
Version 0.1 | Concept note / position statement; not peer reviewed
Draft date: 14 October 2026
Status: Unpublished local-review version

Abstract

The future personal computing environment may take the form of an AI coordination layer over existing software rather than another comprehensive application. This note proposes AI-Mediated Personal Infrastructure (AMPI): a design framework that retains tools' specialized capabilities and coordinates them around a person's goals through semantic interpretation, tool invocation, persistent state, and verifiable execution. AI helps decide what information means and which system should perform a task. Conventional programs retain responsibility for bounded access, durable records, permissions, execution checks, and recovery. AcademicWorkspace, the author's privately operated academic workflow environment, grounds the framework in a concrete practice. A sanitized historical archiving episode illustrates how source identity, user decisions, and generated interpretations remain distinct. AMPI includes direct interfaces, command-line tools, and optional GUI agents; it requires no particular operating system, model, or new interface. Its contribution is an explicit synthesis of design commitments and an implementation-informed account, building on substantial prior work. The single-person case establishes neither general effectiveness nor historical priority.

1. The problem and a definition

A personal task often spans systems that each work well independently. A learning platform holds requirements, an editor holds a draft, a device supplies a recording, and a file collection preserves completed work. Someone must still reconcile identities, interpret a notice, locate the relevant version, and determine whether an operation actually succeeded. This coordination burden persists even when AI makes application development easier.

AMPI treats the person's continuing work as the organizing unit. A task can outlive a chat session and cross application boundaries while retaining its evidence, decisions, and artifacts. The design preference is to compose useful capabilities before rebuilding them. A graphical interface remains welcome when it serves the task; a new interface is not the necessary product of every integration.

> **AI-Mediated Personal Infrastructure is a persistent, user-directed computing environment in which replaceable AI capabilities coordinate the meaning and use of existing digital systems, while conventional components preserve state, enforce scoped authority, execute permitted operations, and support verification and recovery.**

This is a proposed architectural framing. It is neither a new reasoning algorithm nor a certification that any named implementation satisfies every principle. Its central allocation of responsibility is simple: **AI should know which existing system should do the work; it need not perform every step itself.**

2. Design commitments

Preserve specialized capabilities. Source applications remain responsible for their domain functions. Editors continue to edit; storage systems preserve files; conventional clients retrieve authorized data. The coordination layer should use a supported interface or established command when it can express the operation adequately. Integration still requires maintenance, mappings, and permission checks.

Separate interpretation from execution. AI can distinguish a requirement from an example, reconcile terminology, or propose an association between sources. Its output is a candidate interpretation or plan. Programs can check identifiers, input shape, allowed operations, file hashes, and preconditions before applying it. Deterministic means that these checks follow explicit procedures; it does not mean that code, source data, or the chosen procedure must be correct. A valid JSON object can still express the wrong task.

Separate kinds of authority in durable state. A platform observation, a user's completion judgement, and an AI suggestion answer different questions. Preserve them as separate fields, with source references and observation times. An inferred deadline should not silently replace a platform deadline. A task marked complete locally should not imply remote submission. Chat history may help with interpretation, but it should not be the sole store of operational truth.

Make evidence and coverage inspectable. Keep enough provenance to reconstruct why a conclusion was made: source identity, acquisition time, scope, relevant location, and unresolved contradictions. Report inaccessible or unchecked sources explicitly. A successful connector call proves only what that call covered; a current retrieval time does not make the underlying cached data current. Retain necessary evidence without turning the personal system into an indiscriminate archive.

Bind authority to concrete operations. Reading, producing a draft, modifying a user's file, and changing an external platform have different consequences. Reviewable plans should name the target, intended effect, and exact artifacts, with preconditions rechecked before consequential execution. A confirmation string is an operational convention, not cryptographic proof that a human authorized the action. Enforcement depends on the executor, its privileges, and every route by which an operation can be invoked.

Protect human work and support recovery. Distinguish generated files from originals and user-edited artifacts. For replacement, use a trustworthy previous baseline, detect intervening edits, and preserve a recoverable version. Equality of hashes establishes byte identity, not ownership or intellectual correctness. Unknown ownership or conflicting edits should stop automatic replacement. Retrying an ambiguous remote operation should first check the remote result, since repetition can duplicate a side effect.

Make AI substitutable at an explicit boundary. Persist task inputs and results independently of a provider's conversation format. An adapter can then substitute a model or defer semantic work while ordinary acquisition and file processing continue. This portability has costs: tool protocols, context packaging, schema behavior, permissions, and interpretation quality need renewed validation. Replacement is an engineering capability, not a promise of equal answers from interchangeable models.

3. A reference architecture

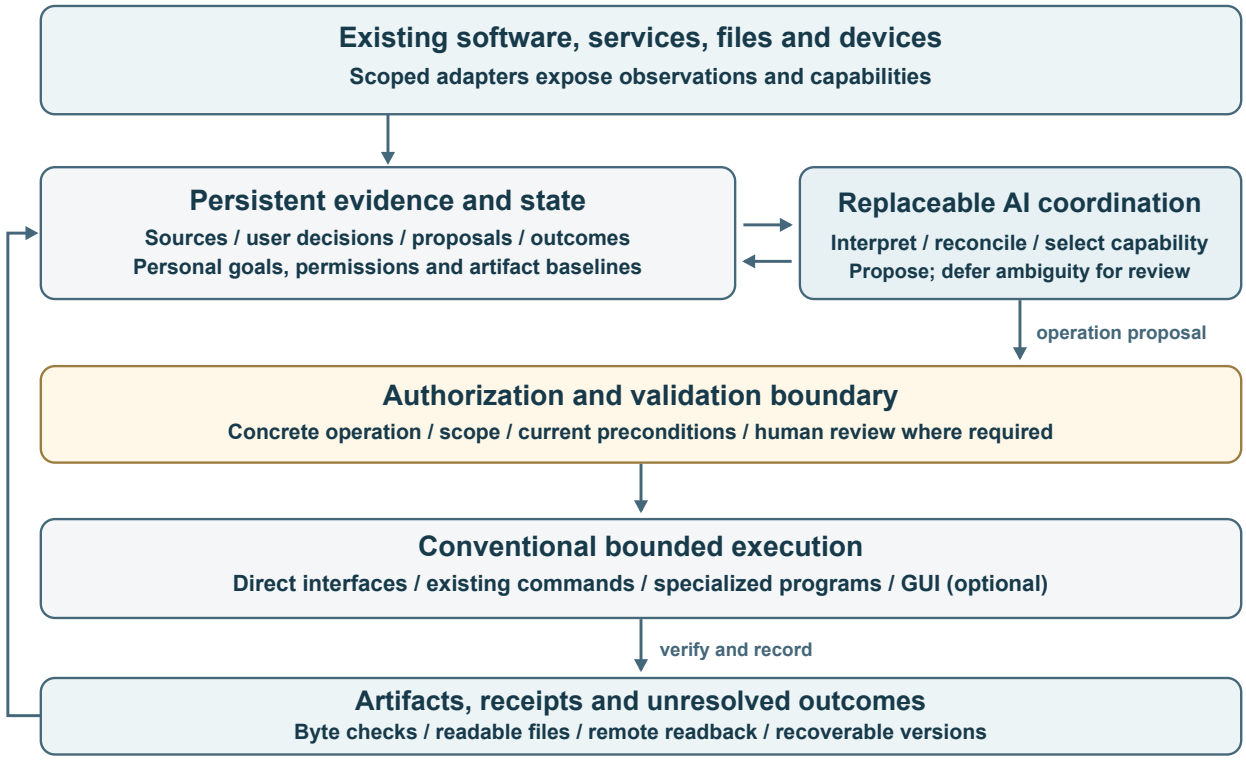


Figure 1. Original conceptual diagram. The GUI executor is optional. This is a reference architecture, not a deployment screenshot or a claim that all controls are universally enforced in AcademicWorkspace.

The architecture has five cooperating responsibilities. **Source adapters** expose scoped observations and existing capabilities. **Persistent records** retain task identity, provenance, user decisions, proposals, and execution evidence. A **semantic coordinator** interprets these records and selects an appropriate capability. An **authorization and validation boundary** checks the proposed operation against its scope and current preconditions. **Conventional executors** perform the bounded action, after which file checks or remote readback establish an outcome.

The arrows form a feedback loop. A coordinator may request another observation or ask the user to resolve a conflict. Execution should distinguish proposed, permitted, attempted, verified, and unresolved outcomes. These are reference distinctions; existing systems may use different state names or only implement part of the separation.

The persistent records need not be a new universal database. Files, small structured stores, and established platform records can cooperate if their roles are explicit. Likewise, verifiable execution does not require recording an AI's internal reasoning. Source evidence, decisions, inputs, operations, and observed outcomes are the useful accountability record. Source content supplies evidence, not permission to execute instructions embedded within it.

4. AcademicWorkspace: a situated implementation

AcademicWorkspace is the author's private, locally operated academic work environment. It combines existing source clients, ordinary files and editors, structured task records, model-facing queries, and a protected artifact publisher. The engineering workspace, long-term materials collection, and task working copies have separate roles. This note describes the current working-tree implementation inspected on 14 October 2026, including local changes absent from its main checkout's HEAD. No private code, course content, or operational snapshot accompanies the publication.

The task store separates remote observations, user fields, AI suggestions, and conflicts. Refreshing a platform item preserves a user's completion mark and edited title. The publisher stages a file, checks its hash, checks whether the target still matches a trusted baseline, and replaces it through a local atomic operation. Identical content is reused; edited or untracked targets can produce a conflict. These checks apply to callers using that publisher, not every program that can write to the file system.

A separate MCP facade exposes bounded, scoped reads of local snapshots; remote submission uses a different command path. The semantic adapter saves input bundles before calling a provider and supports codex, claude, and none routes. The none route leaves semantic work pending, while conventional operations remain available. These interfaces make substitution possible, but their existence establishes neither equivalent interpretation quality nor current authentication of every provider.

A complete, sanitized task chain

A private record dated 1 October 2026 documents an assignment submitted through another client but missing from the local materials collection. The following account preserves the mechanism and removes the course, filename, content, grade, account, paths, and platform identifiers.

1. **Identify the task and source.** Within the authorized review, the operator used the existing platform adapter to relate an assignment to the author's actual submission record. The coordination task was to reconcile a historical local requirement with a newer remote observation, rather than infer completion from a similar filename.

2. **Select and acquire the correct artifacts.** The existing downloader retrieved the submitted document and a separate feedback attachment. The operator kept the platform relationship and retrieval evidence with the archive. A locally edited draft was not substituted for the remote submitted version, and the platform did not establish which other client had been used.

3. **Validate and publish locally.** The publisher copied the acquired artifacts into the established collection with hash checks. The historical validation record reports successful document decoding and visual inspection. Feedback was preserved in its actual data format, rather than represented as a newly rendered, marked-up document.

4. **Record meaning without rewriting history.** A readable submission note linked the artifacts and distinguished the old local snapshot from the observed remote submission. Older indexes lacked trustworthy publishing baselines and remained in conflict; their original contents were preserved. Artifact publication therefore succeeded while some index refreshes did not.

5. **Check replay and authority.** Republishing the same submitted file returned unchanged. The historical record reports unchanged existing assignment files and no remote write or change to the

user's local task status. The archived bytes and provenance, rather than a conversational claim of success, were the deliverable.

This was an Agent-operated workflow using conventional components, not evidence of an unattended service that automatically discovers and resolves every academic task. The historical receipts were inspected for this note; the episode was not repeated against the live platform. Current isolated checks of selected code bodies reproduced state preservation, protected publication, replay, and plan revalidation using synthetic inputs. They do not constitute a production end-to-end test or independent audit.

Separate submission code supports a concrete prepare-and-confirm plan, file and account revalidation, a default-closed write gate, and subsequent platform readback. The inspected code can nevertheless label an attempted submission internally as submitted while readback remains inconclusive. Similarly, the semantic adapter normalizes provider output but does not uniformly perform local business-schema validation. These gaps illustrate why AMPI's verification commitments must be assessed beyond internal status names and documentation.

The author reports that routine use avoids manually opening each academic platform, saves time, and entails manageable maintenance. This is an author account of use, without measured time savings, reliability rates, controlled comparison, or evidence of improved learning outcomes.

5. AMPI and OS-level GUI agents

An OS-level GUI agent interprets visible interface state and acts through interactions such as taps, clicks, and text entry. It can provide access where a suitable direct interface is absent. A product can also combine this route with system tools and service interfaces. For example, the official Doubao Phone Assistant description available during this review explicitly includes simulated interaction, tool invocation, and standardized service interfaces [9]. Treating it as a purely screen-clicking system would misstate that description. No hands-on assessment of the product was conducted here.

AMPI and GUI agents address overlapping work at different architectural scopes. GUI interaction is one possible acquisition or execution channel; AMPI organizes the continuing relationship among goals, sources, authority, state, and artifacts. A GUI agent can participate in that infrastructure. A sufficiently persistent, governed GUI-agent system may already embody several AMPI commitments. The terminology does not establish a mutually exclusive product category.

The proposed routing principle is: **choose the existing capability that can express an authorized operation with suitable observability and recovery, and use GUI interaction when it is needed.** In practice, prefer a supported direct interface or established command for a structured retrieval, hashing operation, or file publication. Reuse a specialized program for its domain operation. Use a GUI agent when the necessary capability is genuinely available only through the interface. A visible review screen may also be preferable when it exposes important context missing from a direct interface.

This preference is conditional. An API can be unstable, incomplete, overprivileged, or disallowed; it is not inherently safer than a GUI. A GUI can make a consequential choice visible, yet remain brittle under layout changes, transient dialogs, focus errors, and ambiguous responses. Both routes need scoped permission and outcome checks. A completed click sequence alone does not establish that the intended file was saved or the external operation succeeded.

AcademicWorkspace does not depend on a GUI agent to maintain task state or preserve files. Its inspected semantic adapter and conventional publishers illustrate delegation across systems; they do not demonstrate integration with Doubao Phone Assistant. The architectural claim is composability of execution channels, not interoperability already implemented between these two systems.

6. Related work and the contribution's scope

MRKL already combines neural language capabilities with discrete modules and external knowledge [1]. ReAct interleaves language-model reasoning and environment actions [2]. MCP standardizes context and tool connections while explicitly leaving enforcement of its security principles to implementers [3]. Temporal documents persistent event histories and replay for recovering workflow execution [4]. AMPI inherits this broad separation and composition of responsibilities; it does not introduce tool use or durable workflows.

Personal infrastructure also has close precedents. Miessler's preserved 2025 PAI account describes personalized skills, tool composition, and a preference for deterministic code and existing commands [5]. Kojukhov, Levin, and Bovshover explicitly describe semantic mediation across heterogeneous agent outputs [6]. AgentOS proposes personal computing organized around intent, skills, and knowledge discovery, including a natural-language interface and agent kernel [7]. SemaClaw addresses personal agents through a harness containing orchestration, permissions, context, and knowledge-management mechanisms [8]. These precede this note and overlap substantively with its motivation.

AMPI's emphasis is a conservative integration framework grounded in the author's academic practice: retain specialized software and ordinary editable artifacts; distinguish platform facts, personal judgements, and AI proposals; bind consequential actions to reviewable plans; and treat preservation, readback, and recovery as continuing responsibilities outside a model session. It does not require replacing the operating system or establishing a new central application. These are emphases and a situated synthesis, not proof that related systems lack them.

AI-assisted development of an ordinary application can produce AMPI components or a complete AMPI system. An assistant inside one application can contribute domain knowledge and actions. Fixed automation is preferable for stable, unambiguous steps, and remains part of AMPI. A general tool-using agent can provide the semantic coordinator; it becomes infrastructure through its relationship with maintained state, bounded execution, and recoverable artifacts. The useful comparison is where responsibilities reside and how their boundaries are tested.

The literature search was focused rather than systematic. Exact terminology and nearby designs were checked, but no claim is made that the label is unused, that the synthesis is globally first, or that a different acronym implies a distinct technical invention.

7. Conditions, failure modes, and evaluation

AMPI is useful to explore when work repeatedly crosses systems, reusable capabilities already exist, source scope can be established, and a person can maintain the necessary records and adapters. A single well-integrated application may already meet the need. Where reliable access, recoverable state, or meaningful verification cannot be established, the coordination layer may add more uncertainty than value. Strict latency requirements and consequential decisions requiring specialist judgement demand additional assurance.

Failures include mistaken cross-source associations, stale caches, malicious instructions in source content, overbroad credentials, model interpretations that pass syntax checks, damaged publishing baselines, concurrent file edits, and duplicated remote actions after a lost response. Recovery records can themselves expose private information or preserve an incorrect conclusion. Some controls reside in code; others depend on operator discipline. Their coverage must be examined individually.

Evaluation should compare completed tasks with the person's existing workflow while measuring interpretation errors, missed requirements, navigation effort, preservation conflicts, maintenance time, and unverified outcomes. Record unavailable sources and failed attempts, not only successful automations. Test loss of a response before retry, restoration of state and artifacts on a clean environment, and provider substitution under the same task and evidence constraints. Learning benefit requires separate evaluation; archival completeness and a submitted document do not establish understanding.

8. Author contribution, assistance, and citation

Omica Chow contributes the architecture's synthesis and design choices, its situated implementation and use in AcademicWorkspace, and this explicit formulation of AMPI. Existing libraries, clients, models, and the cited ideas remain attributable to their creators. The proposed definition and the account of implementation are the contribution documented here; scalability, general benefit, and stronger assurance remain research questions.

The term AMPI arose in discussions between the author and ChatGPT about existing practice. This manuscript was prepared with AI assistance for research support, code inspection, organization, writing, translation, and production. AI is not an author. Omica Chow is responsible for reviewing and approving the public version and its claims.

Citation of this local version: Chow, Omica. *AI-Mediated Personal Infrastructure: From AI-Built Apps to AI-Coordinated Personal Systems*. Version 0.1. Concept note / position statement. Unpublished local-review version.

The draft date identifies preparation. This version has no public publication date or assigned DOI. A public release can provide an attributable, dated record of this formulation; its timestamp does not settle worldwide priority or confer exclusive ownership of a general idea. Machine-readable citation metadata accompanies the manuscript.

References

1. Karpas, E., et al. (2022). *MRKL Systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning*. arXiv:2205.00445, version 1, 1 May 2022. Original paper.

2. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023; arXiv:2210.03629, version 3. First preprint: 6 October 2022. Original paper.

3. Model Context Protocol. (2025). *Specification*, revision 2025-06-18. Official specification.

4. Temporal Technologies. (n.d.). *Temporal Workflow Execution overview*. Official documentation, accessed 14 October 2026. Workflow execution and replay.

5. Miessler, D. (2025). *Building a Personal AI Infrastructure (PAI) (December 2025 Version)*. Preserved December revision of a post dated 26 July 2025. Author's account.

6. Kojukhov, A., Levin, I., and Bovshover, A. (2025). *From Subsumption to Semantic Mediation: A Generative Orchestration Architecture for Autonomous Systems*. Algorithms, 18(12), 773. Publisher DOI.

7. Liu, R., Zhe, T., Wang, D., Yao, Z., Liu, K., Fu, Y., Liu, H., and Pei, J. (2026). *AgentOS: From Application Silos to a Natural Language-Driven Data Ecosystem*. arXiv:2603.08938, version 2, 11 March 2026. Original preprint.

8. Zhu, N., et al. (2026). *SemaClaw: A Step Towards General-Purpose Personal AI Agents through Harness Engineering*. arXiv:2604.11548, version 1, 13 April 2026. Original preprint.

9. Doubao Phone Assistant team. (n.d.). 豆包手机助手官方网站 *[Doubao Phone Assistant official website]*. Product description, accessed 14 October 2026. Official website.

中文摘要：AI 中介式个人基础设施

未来的个人计算环境不一定需要另一个超级应用，也可能是一套 AI 协调层：保留现有软件各自的专业能力，通过语义理解、工具调用、持久状态和可验证执行，将它们整合为以个人目标为中心的基础设施。本文把这一设计框架称为 AI 中介式个人基础设施（AI-Mediated Personal Infrastructure，AMPI）。AI 负责理解资料、关联情境和选择能力；常规程序负责受限访问、状态保存、授权检查、具体操作、结果校验与恢复。AI 更应判断哪些事情交给哪个现有系统完成，不必亲自执行每一个步骤。

AMPI 的组织中心是跨越多个软件、持续存在的个人任务及真实成果。平台观察、用户判断和 AI 建议分别保存；聊天记录不充当唯一数据库，推断不静默覆盖来源事实。自动生成文件与人工编辑区保持边界，替换依赖可信基线，冲突保留原稿。确定性程序也可能出错，语法有效、工具返回成功、内部状态标记与实际结果必须区分。

AcademicWorkspace 提供一个具体实践。本文描述一条经脱敏的历史任务链：识别从其他客户端提交的作业，取回实际提交文件与批改附件，校验后归档，保存来源并重放验证。缺少可靠基线的旧索引保持冲突，没有为归档而重新提交或修改用户任务状态。这是 Agent 编排已有组件的操作记录，不代表已部署无人值守、全来源自动融合服务。当前代码检查也发现统一业务校验与内部提交状态仍存在保障缺口。

AMPI 可以采用直接接口、现有命令、专业程序或 GUI Agent。它接纳 GUI Agent，也能独立于 GUI Agent 运作。正文以豆包手机助手的官方说明讨论这种关系：该产品本身结合模拟操作、工具调用和服务接口，不能把它简化为纯点击系统；本文也未实测该产品或完成与 AcademicWorkspace 的接入。通道选择应依据授权范围、可观察结果和恢复条件，直接接口并不天然更安全。

本文贡献在于作者对个人持久状态、工具组合、来源证据、人工资料保护与授权执行的具体综合和实践阐述。MRKL、ReAct、PAI、语义中介架构、AgentOS、SemaClaw 等已有工作与它实质重合。本文不宣称全球首创；单人使用体验没有提供时间节省、可靠率或学习成效的对照测量。术语与文稿有 ChatGPT / AI 辅助，作者 Omica Chow 负责审核。本文为未经同行评议的概念说明；草稿日期不等于公开发表日期。